Domain LLM Fine-Tuning — Team Guide

Project	Via Codos Proof of Concept (VC4B) ERDP · Jira project `KAN`
Sprint	Mon 17 Aug – Tue 1 Sep 2026 (12 working days)
Team lead	Kusal Pabasara
Version	1.0 · drafted 17 Aug 2026

Read this first

None of us have built a model like this before. This guide is the shared map: what we're building, why, and what to do on each of the twelve days. Jira has the tickets — this has the reasoning behind them.

If you read only one section, read What we are actually building and The five rules. Everything else you can come back to on the day you need it.

Contents

What we are actually building

Strip away the day labels and this is **one controlled experiment, run four times in parallel.**

Each of us takes two base models — **Qwen** and **Llama** — and fine-tunes them on our own industry's data, four times over, with each round using better data than the last. Then we measure whether the data changes actually made the models better.

VERSION	DATA IT TRAINS ON	THE QUESTION IT ANSWERS
v1	Real collected data, cleaned	Baseline. What do we get for free?
v2	v1 + ~500 synthetic examples targeting v1's weak spots	Does targeted synthetic data fix known gaps?
v3	Same data restructured with a `context` field	Can the model learn to read retrieved passages?
v4	RAG-aware, evaluated with live retrieval	Does the full pipeline beat a bare model?

Two models × four versions = **8 trained artifacts per person**, 32 across the team.

The deliverable is not a model. It is a comparison. On Day 12 each of us produces a report saying which model performs better for our industry, backed by numbers. Every day before that exists to produce a row in that final table.

Why four industries

Running the same method across Healthcare, Finance, SME, and Retail lets the company compare *domains*, not just models. That only works if we run the method **the same way**. Where this guide says "agree as a team", that's why.

Who owns what

Everyone runs the same 12-day shape, on the same dates, with the same due dates.

LANE	OWNER	MODELS	JIRA RANGE
Healthcare	**Kusal Pabasara**	Qwen-Healthcare, Llama-Healthcare	KAN-11 → KAN-59
Finance	**Navodya Dissanayake**	Qwen-Finance, Llama-Finance	KAN-12 → KAN-60
SME Daily Business	**Deepana Nirmal**	Qwen-SME, Llama-SME	KAN-13 → KAN-61
Retail / E-commerce / Manufacturing	**Rimaz Nowfel**	Qwen-RetailEcomManufacturing, Llama-...	KAN-14 → KAN-62
Retail lane support	**Thisal Sooriyanayaka**	—	Days 3, 4, 8, 9

Because the lanes are synchronised, **any problem you hit on Day N, three other people hit the same day.** Post fixes in the team channel — ten minutes of writing saves the team hours. This is the single highest-value habit for this sprint.

Reading the Jira dates correctly

Due dates are set to `12:00 AM` of the named day, which means the **start** of that day. Day 1's "due 17/Aug 12:00 AM" means the work happens **on the 17th** and must be done before the 18th begins. Not "due at the end of the 17th".

The five rules that protect the experiment

Break any of these and the final comparison stops being trustworthy. They cost nothing to follow on Day 1 and are expensive to fix later.

1. `data/raw/` is read-only for the whole sprint. Everything else derives from it. If you lose or modify your raw data, v1 can never be reproduced and the whole version comparison collapses.

2. Fix the split seed on Day 2 and write it down. Your 80/10/10 train/validation/test split must be identical across all four versions. A reshuffled test set makes v1 and v4 scores incomparable — you'd be measuring different exams.

3. Do not touch the test split until Day 11. If you look at test results and tune against them, your final numbers describe how well you tuned, not how good the model is. Use the validation split during development.

4. Write the evaluation harness on Day 3, then freeze it. All 8 of your artifacts must be scored by *identical* code. If you improve the harness on Day 8, every earlier score is invalid and you re-run everything on Day 11.

5. Version changes are config edits, never code edits. `train_v2.json` → `train_v3.json` should be a line in a config file. If you're editing the training script between versions, you've changed two things at once and can't attribute the difference.

Concepts you need, in plain terms

Skip anything you already know.

Fine-tuning

Taking a model that already understands language and teaching it your domain. We are not training from scratch — that costs millions. We're adjusting an existing model.

LoRA and QLoRA

Fine-tuning normally means updating billions of parameters, which needs far more GPU memory than we have.

LoRA (Low-Rank Adaptation) freezes the original model and trains a small set of extra weights — an "adapter" — that sits alongside it. You train maybe 0.1% as many parameters.

QLoRA adds quantisation: the frozen base model is compressed to 4 bits, cutting memory further. This is what makes a 7B model trainable on a free Colab T4.

Our settings, the same for everyone: **rank 16, alpha 32, 4-bit, 3 epochs.**

> **What you actually save** is the adapter — a few hundred MB — not the full model. To use it later you load the base model and apply the adapter on top.

Instruction–response pairs

The format models learn from. Raw data becomes:

```
{"instruction": "What is the first-line treatment for X?",
 "input": "",
 "output": "The first-line treatment is..."}
```

Day 2 is mostly converting your raw data into this shape.

Epochs, loss, checkpoints

- **Epoch** — one full pass through your training data. We use 3.
- **Loss** — how wrong the model is. It should go *down*. If it doesn't, something's broken.
- **Checkpoint** — a saved snapshot mid-training. Essential on Colab, which disconnects.

RAG (Retrieval-Augmented Generation)

Instead of relying on what the model memorised, you *look up* relevant documents at question time and hand them to the model as context. Days 7–10 are about making our models good at using retrieved context.

ChromaDB is the vector database we use to store and search documents by meaning.

BLEU and ROUGE

Metrics comparing generated text to a reference answer by word overlap.

> **Important limitation:** they measure *wording*, not *correctness*. A confidently wrong answer phrased like the reference scores well. This is why manual testing is in every ticket — do not skip it.

Choosing your datasets

Do this before you download anything. Two of the four best-known healthcare datasets turned out to be unusable for a commercial project, and the same trap exists in every lane.

The licence rule

This is a **commercial** proof of concept. That rules out:

LICENCE TYPE	VERDICT	WHY
MIT, Apache-2.0, CC-BY-4.0	✅ Safe	Commercial use permitted, attribution only
CDLA-Sharing, CC-BY-SA	⚠️ Ask first	Share-alike may encumber derived model weights
CC-BY-NC (any)	❌ Do not use	Non-commercial. Explicitly excludes this project
No licence stated	❌ Do not use	No licence means no permission

Check the licence on the dataset's own page. **Do not trust a blog post or a tutorial** — several widely-recommended datasets are non-commercial and the tutorials never mention it.

Verified starting points per lane

Checked against dataset pages on 17 Aug 2026. Verify again before you use them.

Healthcare — Kusal

- `openlifescienceai/medmcqa` — Apache-2.0 ✅ — 182k medical MCQs, 21 subjects, expert explanations
- `qiaojin/PubMedQA` (pqa_labeled) — MIT ✅ — 1k expert-labeled QA **with a `context` field**
- ❌ ChatDoctor-HealthCareMagic-100k — no stated licence
- ❌ MedQuAD — CC BY-SA 4.0 share-alike

Finance — Navodya

- `ibm-research/finqa` — CC-BY-4.0 ✅ — 8k QA over financial tables, numerical reasoning
- `gbharti/finance-alpaca` — MIT ✅ — 68k instruction pairs

- ❌ **Financial PhraseBank — CC-BY-NC-SA-3.0.** This is the most commonly recommended finance dataset and it is **non-commercial**. The authors will license it commercially on request — ask early if you want it.

Retail / E-commerce / Manufacturing — Rimaz & Thisal

- `bitext/Bitext-retail-ecommerce-llm-chatbot-training-dataset` — CDLA-Sharing-1.0 ⚠️ — 44.8k instruction/response pairs. Commercial use permitted **but share-alike** — confirm with whoever owns licensing before building v1 on it.

SME Daily Business — Deepana This is the hardest lane to source, because "SME daily business" has no standard benchmark. Expect to combine general business-instruction data with customer-service data, and to lean more heavily on synthetic generation from Day 5. Budget extra time on Day 1 for the search, and ask the team if you're stuck — this is a genuine difficulty, not a personal one.

Record everything as you download

Fill in a `SOURCES.md` with dataset name, URL, licence, row count, and date. The Day 12 report needs it, and reconstructing licences two weeks later is miserable.

The twelve days

The same shape for everyone. Dates are when the work happens.

Day 1 — Mon 17 Aug · Environment Setup + Data Collection

Get Colab running with a GPU, install pinned library versions, download your datasets, save raw copies, back up to Drive.

- **Set Runtime → Change runtime type → T4 GPU** before anything else.
- **Pin your library versions** and share them with the team. A silent version bump changes results with no visible cause.
- Keep raw data on **Google Drive**, not `/content` — Colab wipes local storage.

Done when: GPU verified, datasets on Drive, licences recorded.

Day 2 — Tue 18 Aug · Data Cleaning + QLoRA Configuration

Clean and normalise, format into instruction–response pairs, split 80/10/10, write configs for both models, confirm both load in 4-bit.

- **Record your split seed.** (Rule 2)
- Qwen and Llama want **different chat templates and pad-token handling**. This is the most common Day 2 blocker — expect it, and post your fix.
- Proving both models load and generate one token is the real goal here.

Done when: `train.json` / `val.json` / `test.json` exist and both models load in 4-bit.

Day 3 — Wed 19 Aug · Writing Training Pipelines 🏳 highest leverage

Write training scripts for both models, smoke-test on a small subset, set up Weights & Biases.

This day determines whether Days 4–12 are calm or painful. Everything after this re-runs what you write today with a different data path.

- Make the **data path and version tag config-driven** (Rule 5).
- **Write the evaluation harness now**, alongside training — BLEU/ROUGE plus a fixed set of manual questions. Then freeze it (Rule 4).
- Smoke test means 20 examples and 10 steps: does loss decrease, does a checkpoint save?

Done when: both scripts train on a tiny subset without error, and the harness exists.

Day 4 — Thu 20 Aug · Running v1 Training 🖥

Run both trainings, monitor loss and GPU memory, save as v1.

- **Run sequentially, not simultaneously.** Two 4-bit models plus LoRA will exhaust a single T4. Qwen first, then Llama.

- **Checkpoint to Drive every ~100 steps.** Colab disconnects; a drop should cost minutes.
- Record **wall-clock training time** — the Day 12 report needs it.
- Long GPU waits are the natural time to review a teammate's pipeline.

Done when: `qwen-v1` and `llama-v1` are saved to Drive with training times recorded.

Day 5 — Fri 21 Aug · Testing v1 + Synthetic Data ⚑ heaviest day

Score v1, manually probe 20 domain questions, identify weak areas, generate 500+ synthetic Q&A pairs targeting those weaknesses, merge into `train_v2.json`.

Two distinct jobs in one day. Start the manual probing early — the synthetic generation depends on knowing what's weak.

- **The weakness analysis is the real deliverable**, not the 500 pairs. Random synthetic data will not move v2. Write down specific failure categories first.
- **Tag synthetic examples as synthetic** so v2's composition stays auditable.
- Spot-check generated content against a real source, and record that you did.

Done when: you can name your model's specific weaknesses, and `train_v2.json` exists.

Day 6 — Mon 24 Aug · v2 Training + Self-Testing

> Weekend gap before this day. Colab environments break over gaps — budget setup time.

Train both on `train_v2.json`, evaluate, manually test 30 questions, document the v1→v2 change.

- **First real evidence the method works.** If v2 hasn't improved, the synthetic data missed the weaknesses. **Say so plainly** — a negative result documented is worth more than a vague positive.

- Write up the delta the same day, while you remember what the outputs looked like.

Done when: v2 artifacts saved and the v1→v2 comparison is written down.

Day 7 — Tue 25 Aug · Preparing v3 Data (RAG-Aware) 🏳 lightest day

Merge data into `train_v3.json` and add a `context` field to every example.

- **No training runs today.** Use the slack for Day 9 preparation and helping teammates.
- **Context must resemble what ChromaDB will actually return** — messy retrieved chunks, not clean hand-written summaries. Train on tidy context and the model breaks on real retrieval.
- Include examples where context is **irrelevant or absent**, so the model learns not to blindly trust retrieval.
- **Read the ChromaDB docs today** and decide your embedding model and chunking strategy.

Done when: `train_v3.json` has context on every example, and you know your Day 9 plan.

Day 8 — Wed 26 Aug · v3 Training + Self-Testing 🖥

Train both on `train_v3.json`, evaluate, compare against v2.

> **Expect v3 to score flat or slightly worse** on plain BLEU/ROUGE without retrieval. This is **not** a regression — v3 is trained to use context it isn't being given yet. Note the expectation now so nobody "fixes" a result that isn't broken.

Done when: v3 artifacts saved, and the expected dip is documented as expected.

Day 9 — Thu 27 Aug · RAG Integration ⚠ highest risk

Stand up ChromaDB, index your documents as embeddings, wire up retrieve → context → model → answer, evaluate on 200 queries.

The most likely day to slip. New infrastructure, embedding choices, chunking strategy, and 200 evaluation queries — with Day 10 depending on all of it.

- **Get one correct end-to-end answer before optimising anything.** Working beats tuned.
- **Persist the Chroma index to Drive.** Re-embedding after a disconnect costs hours.
- **Sanity-check retrieval on its own.** If the retriever returns junk, no model quality saves the answer.
- **Build `train_v4.json` today** (v3 + real retrieved context) — see Known traps.
- **If you're going to slip, say so today**, while a weekend of buffer still exists.

Done when: the RAG pipeline answers correctly end to end, and the index is on Drive.

Day 10 — Fri 28 Aug · v4 Training + RAG Testing 🗄

Train both on `train_v4.json`, test with RAG attached on 100 queries.

This is the payoff run. v4 + RAG is the configuration your recommendation rests on.

Done when: v4 artifacts saved and RAG-attached results documented.

Day 11 — Mon 31 Aug · Final Validation

> Weekend gap before this day.

Load all 8 artifacts, run the frozen harness across every version, build the comparison chart.

- **This day only works if the harness stayed frozen** (Rule 4).
- Loading 8 adapters takes real time. Start the batch early.
- The chart is the report's centrepiece: version on one axis, metric on the other, two series for the two models. Make it readable by someone who wasn't in the sprint.

Done when: all 8 artifacts scored by identical code, chart built.

Day 12 — Tue 1 Sep · Final Save + Master Report

100 final test questions checked manually. Master report with scores, training times, dataset sizes, and a recommendation. All v4 models uploaded to shared Drive with a registry file.

- **Write the recommendation as a decision, not a summary.** Which model, for your industry, and why — including cost and inference speed, not only BLEU.
- **Include what didn't work.** Failed synthetic categories and the v3 dip are genuinely useful findings. A report with only good news is less trustworthy.
- **Verify uploads open from a different account** before closing the ticket.

Done when: report delivered, models uploaded and verified, registry complete.

Your lane, day by day

Exact Jira issue keys, generated from the board.

Kusal Pabasara — Healthcare

DAY	DATE	ISSUE	TASK
1	17/Aug/26	KAN-11	Environment Setup + Data Collection
2	18/Aug/26	KAN-15	Data Cleaning + QLoRA Configuration
3	19/Aug/26	KAN-19	Writing Training Pipelines
4	20/Aug/26	KAN-24	Running v1 Training (Qwen + Llama)
5	21/Aug/26	KAN-29	Testing v1 + Synthetic Data Generation (v2 prep)
6	24/Aug/26	KAN-33	Running v2 Training + Self-Testing v2
7	25/Aug/26	KAN-37	Preparing v3 Data (RAG-Aware Dataset)
8	26/Aug/26	KAN-41	Running v3 Training + Self-Testing v3
9	27/Aug/26	KAN-46	RAG Integration + Testing
10	28/Aug/26	KAN-51	Running v4 Training + Testing (RAG-Aware)
11	31/Aug/26	KAN-55	Final Validation Across All Versions
12	01/Sep/26	KAN-59	Final Save + Master Report

Navodya Dissanayake — Finance

DAY	DATE	ISSUE	TASK
1	17/Aug/26	KAN-12	Environment Setup + Data Collection
2	18/Aug/26	KAN-16	Data Cleaning + QLoRA Configuration
3	19/Aug/26	KAN-20	Writing Training Pipelines
4	20/Aug/26	KAN-25	Running v1 Training (Qwen + Llama)
5	21/Aug/26	KAN-30	Testing v1 + Synthetic Data Generation (v2 prep)
6	24/Aug/26	KAN-34	Running v2 Training + Self-Testing v2
7	25/Aug/26	KAN-38	Preparing v3 Data (RAG-Aware Dataset)
8	26/Aug/26	KAN-42	Running v3 Training + Self-Testing v3
9	27/Aug/26	KAN-47	RAG Integration + Testing
10	28/Aug/26	KAN-52	Running v4 Training + Testing (RAG-Aware)
11	31/Aug/26	KAN-56	Final Validation Across All Versions
12	01/Sep/26	KAN-60	Final Save + Master Report

Deepana Nirmal — SME Daily Business

DAY	DATE	ISSUE	TASK
1	17/Aug/26	KAN-13	Environment Setup + Data Collection
2	18/Aug/26	KAN-17	Data Cleaning + QLoRA Configuration
3	19/Aug/26	KAN-21	Writing Training Pipelines
4	20/Aug/26	KAN-26	Running v1 Training (Qwen + Llama)
5	21/Aug/26	KAN-31	Testing v1 + Synthetic Data Generation (v2 prep)
6	24/Aug/26	KAN-35	Running v2 Training + Self-Testing v2
7	25/Aug/26	KAN-39	Preparing v3 Data (RAG-Aware Dataset)
8	26/Aug/26	KAN-43	Running v3 Training + Self-Testing v3
9	27/Aug/26	KAN-48	RAG Integration + Testing
10	28/Aug/26	KAN-53	Running v4 Training + Testing (RAG-Aware)
11	31/Aug/26	KAN-57	Final Validation Across All Versions
12	01/Sep/26	KAN-61	Final Save + Master Report

Rimaz Nowfel — Retail / E-commerce / Manufacturing

DAY	DATE	ISSUE	TASK
1	17/Aug/26	KAN-14	Environment Setup + Data Collection
2	18/Aug/26	KAN-18	Data Cleaning + QLoRA Configuration
3	19/Aug/26	KAN-22	Writing Training Pipelines
4	20/Aug/26	KAN-27	Running v1 Training (Qwen + Llama)
5	21/Aug/26	KAN-32	Testing v1 + Synthetic Data Generation (v2 prep)
6	24/Aug/26	KAN-36	Running v2 Training + Self-Testing v2
7	25/Aug/26	KAN-40	Preparing v3 Data (RAG-Aware Dataset)
8	26/Aug/26	KAN-44	Running v3 Training + Self-Testing v3
9	27/Aug/26	KAN-49	RAG Integration + Testing
10	28/Aug/26	KAN-54	Running v4 Training + Testing (RAG-Aware)
11	31/Aug/26	KAN-58	Final Validation Across All Versions
12	01/Sep/26	KAN-62	Final Save + Master Report

Thisal Sooriyanayaka — Support (Retail lane)

DAY	DATE	ISSUE	TASK
3	19/Aug/26	KAN-23	Writing Training Pipelines (Support)
4	20/Aug/26	KAN-28	Running v1 Training (Support)
8	26/Aug/26	KAN-45	Running v3 Training (Support)
9	27/Aug/26	KAN-50	RAG Integration (Support)

Thisal covers Days 3, 4, 8 and 9 — the pipeline-writing day, two GPU-heavy days, and the RAG day. Rimaz and Thisal should agree who runs which model to avoid duplicating work.

Known traps, found the hard way

These were hit during Day 1 in the Healthcare lane. All four apply to every lane.

1. `train_v4.json` has no day assigned to build it

Day 7 produces v3. Day 10 says "train on `train_v4.json`" — but **nothing creates it**. This is a gap in the board, not a misreading.

Do: build it on Day 9 as v3 plus real retrieved context from your Chroma index.

2. bitsandbytes must be 0.46.1 or newer on Colab

Colab ships PyTorch built for CUDA 12.8. **bitsandbytes 0.45.x has no CUDA 12.8 binary** — it imports without error, reports no problem, and silently has no GPU support. You find out when training fails.

```
pip install -U 'bitsandbytes>=0.46.1'
```

Then **restart the runtime.**

The nasty part: constructing a `BitsAndBytesConfig` succeeds even when this is broken, because it's just a config object. The real test is running an `nf4 Linear4bit` matmul on the GPU.

3. Colab recycles VMs between sessions

Drive data survives. **Installed packages do not.** Coming back to a wall of version-mismatch errors usually means a fresh VM, not a broken setup — re-run your install cell and restart.

Budget ~5 minutes at the start of every session for: **pull → install → restart → verify**.

4. `pip install` then verify in the same cell will mislead you

pip writes the new version to disk, but the already-imported module stays in memory for the life of the interpreter. Checking straight after installing tests the **old** module and reports a failure that's already fixed.

Always restart between installing and verifying.

5. T4 does not support bf16

Use `fp16` as your compute dtype. A bf16 config will fail on Colab's T4.

Open questions

Answers needed before Day 5, when the synthetic-data work forces the issue.

1. What is this proof of concept actually demonstrating?
Nobody has told us, and it changes what the Day 12 report argues:

- "*Fine-tuning beats base models*" → headline is v1 vs the untuned base
- "*RAG beats fine-tuning alone*" → headline is v4-with-retrieval vs v1
- "*A small tuned model is good enough*" → cost and speed matter as much as accuracy

Same twelve days of work, different report. **Kusal is raising this with management.**

2. Do we standardise on one evaluation harness across lanes? If all four of us score with identical code, the cross-lane comparison is trivial. If not, someone reconciles four sets of numbers by hand on Day 12. **Decide by Day 3**, while the harness is still being written.

3. Is BLEU/ROUGE enough? They measure word overlap, not correctness. Each lane probably needs one correctness-based measure suited to its domain — exact match where answers are checkable, manual review otherwise. **Also a Day 3 decision.**

Getting help

- **Post blockers in the team channel the same day.** Synchronised lanes mean your Day-N problem is three other people's Day-N problem.
- **Kusal is available for reviews**, particularly during GPU waits on Days 4, 8 and 10.
- **A negative result is a real result.** If v2 doesn't beat v1, that's a finding — document it rather than hiding it.
- **Nobody here has done this before.** Asking early is cheaper than debugging alone for a day.

Reference implementation: `github.com/KusalPabasara/healthcare-llm-finetune` *— the Healthcare lane's scripts, Colab notebook, and environment checks. Reusable across lanes; the only lane-specific parts are the dataset choices.*